Fase Explorar

En esta fase explicaremos cómo hemos avanzado nuestros conocimientos en el sector en el que estamos trabajando, tanto como conectar conceptos, como crear nuevos tipos de soluciones y estar informados para saber realizar la materialización del proyecto de forma adecuada, concisa y correcta.

(Tenemos en GitHub un archivo con links de referencia!)

Raul Martin | Jorge Sanchez | Jorge Salvador

ÍNDICE

- Etapa 1 - Investigación de qué queremos buscar
    - Búsqueda Antes de iniciar la Materialización
    - Conectividad de NetBeans y MySQL
- Etapa 2 - Investigación de qué queremos hacer
    - Sistemas, Lenguaje y Entorno
    - Búsqueda Durante la Materialización
- Etapa 3 - Resumen de Exploración e Inicio de Proyecto
    - Bibliografía

Etapa 1 - Investigación de que queremos buscar

¿Búsqueda Antes de iniciar la Materialización?

Para el concepto general hemos usado principalmente material digital ya que no sabemos leer libros.

Debemos diseñar antes de tiempo como van a ser las clases que vamos a poner en el programa, las tablas que vamos a emplear en la Base de Datos, cómo vamos a unificar el servidor y también cómo realizar el instalable, conocer también cómo hacer un ejecutable tanto en windows como en linux y que funcionen en ambos casos, etc.

Es por eso que en esta sección vamos a poner todo aquello que queremos saber sin aun ejecutarlo:

Uso de GitHub: estructuras de ramas, flujos de trabajo y organización mediante proyectos

Exploración realizada por Jorge Sánchez a base de usar Git y GitHub y sus varias características en proyectos propios, como éste.

Crear un .gitignore y un .gitattributes correctos

Hemos terminado utilizando un par de webs (ésta y ésta otra) que generan una estructura base para cada tipo de archivo de configuración respectivamente a partir de unas etiquetas descriptivas del entorno de trabajo (netbeans, maven, PDFs, fotos, archivos markdown, etc.) que posteriormente hemos revisado y modificado para terminar de adecuar al proyecto.

Uso de Java with Maven y JUnit

Algo de información previa principalmente de videos como éste.

Búsqueda para saber cómo funciona Filezilla y Samba

Hemos buscado en Youtube :

- Como utilizar Filezilla server para poder tener un servidor ftp, Como poder conectarnos desde Filezilla Client o desde cmd
- A parte hemos buscado la manera de poder instalar y configurar para poder pasar archivos Samba desde linux a Windows.

Perfeccionamiento de la base de datos gracias a la IA

Gracias a unos Prompts y unas fotos de la base de datos nos ayudó a la perfección de la base de datos, haciendo algún cambio de relaciones, Cardinalidades o añadiendo más atributos.

Conectividad de NetBeans y MySQL

Para realizar la conectividad de NetBeans y MySQL vimos un video que estará linkado en el apartado de Bibliografía, este explicaba fácilmente como conectar y verificar la unión.

Jorge Salvador creó un nuevo proyecto en consola para ver si se podía hacer la conexión, una vez unificadas creó un método (Conexión Correcta) con un “Sout” que ponía si se había conectado correctamente y una excepción de MySQL por si no se conectaba con “sout” también.

Finalmente tras prueba y error y la información obtenida pudo conectar MySQL con NetBeans para así comenzar a hacer prácticas en CRUD que se explicarán más adelante en el documento (Búsqueda mientras la materialización).

```java
public static Connection conectar() {
    Connection con = null;
    try {
        con = DriverManager.getConnection(url, user, password);
        System.out.println("Conexión exitosa!");
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return con;
}
```

Etapa 2- Investigación de qué queremos Hacer

Sistemas, Lenguaje y Entorno

Sistemas

En principio queremos hacer despliegue tanto a *Windows* como a *Ubuntu Linux*.

Lenguaje

Hemos decidido utilizar *Java*, la opción más obvia dado que es el lenguaje que hemos aprendido a utilizar en clase.

Para la parte gráfica utilizaremos la librería *Swing* de *JavaFX*.

Entorno

El IDE de elección va a ser *NetBeans* y el gestor de proyectos va a ser *Maven* siempre que sea posible, lo que nos facilita el uso de *JUnit* para tests y el control de dependencias mediante archivos de configuración *pom.xml*. Además hacemos control de versiones con *Git* y en remoto con *Github*.

Búsqueda durante la materialización

En este caso hablamos de la búsqueda de información dentro de la etapa de materialización, dudas que van apareciendo durante el desarrollo:

Uso de Maven para proyectos de JavaFX

Estamos teniendo problemas para portar dependencias de javaFX desde ant a maven y le preguntamos a Claude (prompt: *portar dependencias import javaFX de ant a maven*) pero no fue de mucha ayuda.

Interfaz para utilizar HTML y CSS en Java

Encontramos DukeScript pero dadas las limitaciones de licencia detrás de una barrera de pago decidimos que **no era buena idea usarlo**.

Java Interface - Swing

Hemos aprendido a usar eventos y crear paneles para mejorar el apartado gráfico y la interfaz para la comodidad del usuario.

Uso de Generics

Nos surgió una duda, le preguntamos a Claude y nos propuso el uso de *Generics*, así que investigamos al respecto en páginas como StackOverflow y la documentación de Oracle.

La conversación con Claude fue ésta:

tengo en java maven una clase juego con private final Jugador jugador1; private final Jugador jugador2; getters y setters y una clase hija juego2. Tambien tengo clases hijas jugadorJuego2. Quiero forzar que en la segunda clase los jugadores sean de clase jugadorJuego2 y poder acceder a ellos y sus metodos

Hay varias formas de resolver esto en Java. Te explico la mejor aproximación usando generics.

Solución con Generics

```java
// Clase base genérica
public class Juego<J extends Jugador> {
    private final J jugador1;
    private final J jugador2;

    public Juego(J jugador1, J jugador2) {
        this.jugador1 = jugador1;
        this.jugador2 = jugador2;
    }

    public J getJugador1() { return jugador1; }
    public J getJugador2() { return jugador2; }
}
```

y la solución implementada se basa en lo sugerido pero no se copió y pegó nada, se aplicó lo aprendido a mano.

Uso de Java with Maven y JUnit

Aún nos falta trabajo de investigación sobre el uso de Maven, sobre todo a la hora de establecer dependencias y usar JavaFX y Swing.

También necesitamos investigar más sobre testing y JUnit y Juniper en concreto ya que es una librería bastante menos intuitiva que la que hemos utilizado en Entornos para hacer pruebas unitarias en C#.

Uso de hilos para los juegos en relación a la secuencia de ejecución en Swing

Nos falta investigación también sobre las necesidades de uso de hilos para poder ejecutar tanto la UI en Swing como el juego en curso de manera dinámica.

Sistema CRUD con MySQL

Una vez hecha la base de datos Jorge Salvador comenzó a hacer un sistema Crud a través de metodologías en consola:

```java
public void insertarDato(int id, int piedra, int papel, int tijera) {
    String query = "insert into ppt (id_ppt, piedra, papel, tijera) values (?,?,?,?)";
    try {
        Connection con = Conector.conectar();
        PreparedStatement ps = con.prepareStatement(query);
        ps.setInt(1, id);
        ps.setInt(2, piedra);
        ps.setInt(3, papel);
        ps.setInt(4, tijera);
        ps.executeUpdate();

        System.out.println("Dato insertado con exito");
    } catch (SQLException e) {
        System.out.println("Error al insertar Dato");
        e.printStackTrace();
    }
}

public void leerDato() {
    String query = "SELECT * FROM ppt";
    try {
        Connection con = Conector.conectar();
        PreparedStatement ps = con.prepareStatement(query);

        ResultSet rs = ps.executeQuery();

        while (rs.next()) {
            int id = rs.getInt("id_ppt");
            int piedra = rs.getInt("piedra");
            int papel = rs.getInt("papel");
            int tijera = rs.getInt("tijera");
            System.out.println("ID:" + id + " Veces tiradas PIEDRA: " + piedra + " Veces tiradas PAPEL: " + papel + " Veces tiradas TIJERA: " + tijera);
        }

        System.out.println("Dato leido con exito");
    } catch (SQLException e) {
        System.out.println("Error al leer Dato");
        e.printStackTrace();
    }
```

```java
public void actualizarDato(int id, int piedra, int papel, int tijera) {
    String query = "UPDATE ppt SET piedra=?, papel=?, tijera=? WHERE id_ppt=?";
    try {
        Connection con = Conector.conectar();
        PreparedStatement ps = con.prepareStatement(query);
        ps.setInt(1, piedra);
        ps.setInt(2, papel);
        ps.setInt(3, tijera);
        ps.setInt(4, id);
        ps.executeUpdate();

        System.out.println("Dato actualizado con exito");
    } catch (SQLException e) {
        System.out.println("Error al actualizar Dato");
        e.printStackTrace();
    }

}

public void eliminarDato(int id) {
    String query = "DELETE FROM ppt WHERE id_ppt=?";
    try {
        Connection con = Conector.conectar();
        PreparedStatement ps = con.prepareStatement(query);
        ps.setInt(1, id);
        ps.executeUpdate();

        System.out.println("Dato eliminado con exito");
    } catch (SQLException e) {
        System.out.println("Error al eliminar Dato");
        e.printStackTrace();
    }

}
```

Aquí se pueden ver las metodologías investigadas y de hecho perfeccionadas por la revisión de Claude (IA) y su perfección, añadiendo la líneas “ps.executeUpdate();” con la que se pueden ejecutar las consultas y actualizarlas para el sistema CRUD.

Etapa 3- Investigación de qué queremos Hacer

Bibliografía

https://mrkandreev.name/snippets/gitignore-generator/

FIleZilla

Samba

Conectividad java -> Sql